

Computer Science 40S-007

Final Project

Rummy Card Game

Group Name:

Celeste Lopez Boulton

and

Ruhani Jindal

University of Winnipeg Collegiate

Final Project Report

1. Introduction and Motivation

For the final project, we have decided to create a card game called Rummy. We picked this particular card game because it is not too simple or complex; it suits our programming level. It is also a perfect game to play against a computer, allowing the two of us to code different functions at once.

2. Background and Related Works

a. Related Work: Rummy, the card game

Rummy is a card game that can be played between two or more people. The object of the game is to ‘meld’ your cards into two types of combinations.

Runs: These are consecutive sequences of three or more cards of the same suit.

Sets: These are collections of three or more cards of the same rank (but differing suits)

For example, a run can consist of one six of hearts, one seven of spades, and one eight of clubs. Similarly, a set may consist of three sixes of different suits (examples: hearts, spades, and clubs).

You can also lay off cards in Rummy. Laying off is when a player puts down a card that matches the other player’s meld. For example, if player 1 puts down a set of three eights, player 2 can put down another eight to add to the meld. This is another way for a player to get rid of their cards.

At the start of the game, the cards are shuffled by the dealer and each player is dealt ten cards. The remaining cards are placed in the center of the table to form the stock. The first card of the stock is placed faced up beside the pile to start off the discard pile (this leaves us with two different piles: the stock and the discard pile).

The user will go first, drawing a card from either the stock or the discard pile. Then the player will see if they can create any melded combinations or lay off one (or more) of their cards. If the player cannot make any combinations or lay off a card, then they must discard one of their cards. If the player can meld some of their cards, they will create as many melds as possible and/or lay off a card; after this, the player will discard one of their cards, and it’s the next player’s turn, following the same rules.

The first person to sort their entire hand into the above combinations with a final card to place in the discard pile wins the game.

3. Problem Statement

Our goal was to simulate a rummy game between the player and the computer. We broke down this large problem into smaller problems which we solved one by one, such as figuring out the structure and design of the meld and layoff functions, simulating computer logic, and creating a user-friendly interface without graphics.

4. Solution Strategy and Implementation

We made our online Rummy card game by dividing the development tasks effectively between the two programmers, Ruhani and Celeste. The game followed the rules outlined in the Background section, with both the player and computer taking turns to play. We plan to implement our ideas one step at a time. Here are some of our key components:

a. User Interface + Setup

- We decided to simply use the consol as our user interface rather than graphics so that we could focus more on the coding aspect of the project. In terms of format, we used spaces and headers to create an appealing environment for the user.

b. Computer Moves/Logic Functions

- The goal of the computer game functions is to replicate the result of a player function but in such a way that allows the computer to use logic to make decisions. We allocated the responsibility of the two main computer functions for melds and layoffs to the programmers Celeste and Ruhani, respectively.
- The focus of the programming aspect was to carefully go through each logical step required to search for and then create a valid meld, implementing those decisions in the code. For the meld function, the computer must first search for cards that it may potentially meld; for runs, this means looking for consecutive ranks, and for sets, it means looking for matching suits. Next, it must see if there are three or more cards in that list that may form a valid meld using an if-else statement. Based on this analysis, the computer makes a final decision and either melds or doesn't meld.
- For the computer's layoff function, the computer must first examine the meld's type (either sets or runs). It must then go through all the cards in its hand to see if a particular card may be added to an existing meld. If it finds a card to lay off, the computer will follow through and add that card to the existing meld, removing it from its own hand.

c. Functions to accept player input and perform actions

- The player's meld function requires input from the user for the type of meld (sets or runs). The program will then go to the respective section of the code. Additional input is required to figure out which cards the player would like to meld, entered as a string of numbers. Next, the program must check to see if the player has entered a valid meld by

means of an if-else statement. For runs, the cards are checked for matching suits and consecutive ranks; for sets, matching ranks are verified. If the meld is valid, it is added to the list of current melds.

- The player's layoff function asks the user for input about which card it would like to lay off as well as the meld it would like to add to. Multiple if-else statements are used to verify that the user is inputting valid input and melds. More if-else statements are used to check if the card is a valid lay-off for either a run or set meld. If the card is valid, it is added to the selected meld in the current melds list.

d. Clear Instructions for the players

- We want to make sure that users who've never played rummy can still use the app. At the beginning of the game, a player can input the command 'instructions,' prompting the game's instructions to appear on the screen. We had very clear directions for the player in every move, ensuring they would have a fun game experience even without graphics.

5. Experimental Framework

Our approach to developing the Rummy card game involved a combination of structured planning and iterative development. Initially, we aimed to divide the project based on high-level components, with Celeste focusing on core gameplay logic (deck, dealing, drawing, melding) and Ruhani handling user interface, game flow, and scoring. However, as the project progressed, we found a more effective strategy was to divide tasks at a more granular, functional level.

a. Initial Planning vs. Actual Implementation:

- **Planned:**
 - Celeste: Computer moves, logic functions, card deck, draw/discard, meld detection.
 - Ruhani: User interface, player input, game flow, win detection, scoring.
- **Actual:**
 - We discovered that splitting the project into smaller, more manageable functions allowed for more parallel development and reduced merge conflicts. For example, within the meld detection, Celeste focused on the logic for sets and runs separately. Ruhani worked on functions for sorting, layoff, and then on the computer's layoff and discard logic. More details can be seen in our timeline.
 - This refined strategy allowed us to work more efficiently and address issues more quickly.

b. Development Process:

- **Iterative Development:** We adopted an iterative development approach, focusing on implementing core functionalities first (deck creation, dealing, drawing) and then gradually adding more complex features (melding, computer AI, scoring).
- **Version Control:** GitHub played a crucial role in our workflow, enabling us to track changes, manage versions, and collaborate effectively. We used branches for individual features and merged them into the main branch after thorough testing.
- **Debugging:** Debugging consumed a significant portion of our development time. We encountered numerous challenges related to card manipulation, meld detection, and computer AI logic. We used print statements, debugging tools within Thonny, and thorough testing to identify and resolve these issues.
- **Testing:** We conducted regular testing throughout the development process, ensuring that each feature functioned correctly before moving on to the next. We also performed user testing with peers to gather feedback and identify areas for improvement.
- **Task Management:** We used Microsoft Outlook to communicate and track progress, ensuring that we stayed on schedule and addressed any roadblocks promptly.

c. Challenges and Lessons Learned:

- **Complexity of Meld Detection:** Implementing the meld detection logic, especially for runs, proved to be more challenging than anticipated. We had to carefully consider various edge cases and ensure that the logic was robust and accurate.
- **Computer AI:** Developing a simple but functional computer AI required careful planning and implementation. We had to balance the need for logical decision-making with the constraints of our development timeline.
- **Debugging Time:** We underestimated the amount of time required for debugging. This highlighted the importance of thorough testing and careful planning.
- **Collaboration:** Working collaboratively on a complex project required effective communication and coordination. We learned the importance of clear communication, regular meetings, and a shared understanding of the project goals.

In summary, our experimental framework involved a flexible and iterative approach, adapting to the challenges we encountered and leveraging version control and collaboration tools to achieve our project goals.

6. Timeline of Activities

Activity	Programmer	End Time
Started the project and made our GitHub repository	Both	January 16
Made the welcome output and instructions for the player	Ruhani	February 4
Created Card Deck and Shuffle Function	Celeste	February 5
Deal Function	Celeste	February 6
Draw and Discard Functions	Celeste	February 10
Players can now restart the game, score tracking is implemented	Ruhani	February 10
Sorting Function	Ruhani	February 10
Win detection	Ruhani	February 10
Meld Detection – sets	Celeste	February 15
Layoff Function	Ruhani	February 16
Meld Detection – runs	Celeste	February 20
Working on Computer Meld Function	Celeste	March 5
Added Computer Layoff, Computer Discard, Fixing Sort Function	Ruhani	March 9
Computer Meld Function Completed	Celeste	March 11
Debugging, fixing things, adding things to improve user experience	Ruhani	March 12
User Testing	Both	March 15
Working on Report and Presentation	Both	March 25

7. Results

We have made fully functional Rummy card game where:

- **The Player Can Play Against the Computer:** The user should be able to play a full game against the computer, making valid moves like drawing, melding, and discarding cards.
- **Computer AI Performs Logical Actions:** The computer will be able to make decisions based on available cards and apply simple strategies to play the game (not at an expert level).
- **Valid Gameplay and Win Condition:** The game will check for valid melds and ensure that the game ends when one player has completed all their melds and discarded their last card.
- **User Interface:** A simple CLI that is intuitive, displaying game progress and prompting for player actions. By implementing this project, we aim to demonstrate our understanding of the learning done in our CS40S class.

8. Conclusions and Future Work

In this project, we successfully developed a functional Rummy card game, achieving our primary goal of simulating a player versus computer card game environment. This project allowed us to apply and expand our programming skills in Python, particularly in areas such as game logic, user interaction, and basic AI implementation.

Achievement of Proposed Goals:

As outlined in our proposal, our initial objective was to create a Rummy game that included:

- **Player vs. Computer Gameplay:** We successfully implemented this, allowing users to play a full game against a computer opponent.
- **Computer AI:** While not an expert-level AI, the computer player demonstrates logical decision-making in drawing, melding, and discarding cards.
- **Valid Gameplay and Win Conditions:** The game accurately checks for valid melds, enforces game rules, and correctly identifies the winner.
- **User Interface:** We created a functional command-line interface that provides clear game updates and prompts for player actions.

We were able to achieve these goals, though certain aspects, like the complexity of the computer's AI, were simplified due to time constraints.

Future Work:

Looking ahead, there are several avenues for expanding and improving our Rummy game:

- **Enhanced Computer AI:** We could implement more sophisticated AI algorithms to improve the computer's decision-making, making the game more challenging and engaging. We could also include the option for the player to play an easy, medium, or hard version of the game, with more sophisticated AI as per difficulty level.
- **Graphical User Interface (GUI):** Transitioning from a command-line interface to a GUI would significantly enhance the user experience, making the game more visually appealing and intuitive. This could include having a visual version of cards being shuffled and dealt.
- **Multiplayer Functionality:** Adding the ability for multiple human players to compete against each other would add a new dimension to the game.
- **More Card Games:** The framework we have developed could be adapted to create other card games, such as Poker, Blackjack, or Cribbage.

- **Advanced Meld and Layoff Logic:** More complex logic could be implemented to allow for more strategic melds and layoffs. We could also include an AI helper for new players who want help in improving their Rummy skills.
- **Improved Error Handling:** We could add more robust error handling to prevent issues that could arise from incorrect user input. We do however, already have a lot of this.

In conclusion, this project was a valuable learning experience, allowing us to create a fully functional Rummy game and providing a solid foundation for future development.

9. External Materials

a. Development Tools:

- Programming Language: Python
- Integrated Development Environment (IDE): We used PyCharm and Thonny for our text editor supporting Python.
- Version Control: GitHub was used for version control, allowing us to collaborate effectively and track changes made by each member.
- Microsoft Outlook: We used this for effective communication, to track each step of our progress throughout the progress.

b. Hardware:

- A computer/laptop for each team member with Python installed.
- Access to the internet for research and collaboration.

c. Additional Learning Resources

- We used learning resources like YouTube as our current knowledge may not be sufficient for this project.
- Sites like w3schools.com and GeeksforGeeks.org to learn about different Python functions and data structures.

10. Appendix

```

"""
@author: Ruhani Jindal and Celeste
Lopez Boulton

For the final project, we have decided to
create a card game called Rummy.

"""

# ***** Start Game ***** #

# welcome message

welcome = "Hello, welcome to Rummy!
Hope you have fun!\n"

welcome += "In the beginning of any
turn, you can input instructions, sort by
rank, sort by card, or click enter to
continue"

# instructions

instructions = "Instructions: \n"

instructions += "Rummy is a card game
that can be played between two or more
people. \nThe object of the game is to
'meld' your cards into two types of
combinations. \n"

instructions += "Runs: These are
consecutive sequences of three or more
cards of the same suit. \n"

instructions += "Sets: These are
collections of three or more cards of the
same rank (but differing suits) \n"

instructions += "For example, a run can
consist of one six of hearts, one seven of
spades, and one eight of clubs.
\nSimilarly, a set may consist of three
sixes of different suits (examples: hearts,
spades, and clubs). \n"

instructions += "You can also lay off
cards in rummy. Laying off is when a
player puts down a card that matches the
other player's meld. \nFor example, if
player 1 puts down a set of three eights,
player 2 can put down another eight to
add to the meld. \nThis is another way
for a player to get rid of their cards. \n"

instructions += "At the start of the game,
the cards are shuffled by the dealer and
each player is dealt ten cards. \nThe
remaining cards are placed in the center
of the table to form the stock. \n"

instructions += "The first card of the
stock is placed faced up beside the pile
to start off the discard pile \n(this leaves
us with two different piles: the stock and
the discard pile). \n"

instructions += "The user will go first,
drawing a card from either the stock or
the discard pile. \nThen the player will
see if they can create any melded
combinations or lay off one (or more) of
their cards. \n"

instructions += "If the player cannot
make any combinations or lay off a card,
then they must discard one of their cards.
\nIf the player can meld some of their
cards, they will create as many melds as
possible and/or lay off a card; \nafter
this, the player will discard one of their
cards, and it's the next player's turn,
following the same rules. \n"

instructions += "The first person to sort
their entire hand into the above
combinations with a final card to place
in the discard pile wins the game. \n"

print(welcome)

# ***** Initializing Cards ***** #

import random

class Card():

    def __init__(self, suit, rank):

        self.suit = suit

        self.rank = rank

    def __repr__(self):

        return f'{self.rank} of {self.suit}'

# Define suits and ranks

suits = ['♠', '♥', '♦', '♣'] # Unicode
symbols for suits

ranks = ['A', '2', '3', '4', '5', '6', '7', '8', '9',
'10', 'J', 'Q', 'K']

# Create the deck

deck = []

def make_deck():

    # Generate the deck of 52 cards (Card
    objects)

    for suit in suits:

        for rank in ranks:

            card = Card(suit, rank)

            deck.append(card)

# Make suits and ranks identifiable

hearts = [card for card in deck if
card.suit == '♥']

diamonds = [card for card in deck if
card.suit == '♦']

spades = [card for card in deck if
card.suit == '♠']

clubs = [card for card in deck if
card.suit == '♣']

```

```
# Create an empty list for the melds
melds = []
```

```
computer_score = 0
player_score = 0
```

```
# ***** Functions and Descriptions
***** #
```

```
# shuffle: shuffles the deck at the
beginning of the game
```

```
def shuffle(deck):
    random.shuffle(deck)
```

```
# deal: deals the computer and player 10
cards each at the beginning of the game
```

```
def deal(deck):
    if len(deck) == 0:
        make_deck()
```

```
    player_hand = [] # create a new list
    containing the hand of the player
```

```
CARDS_PER_HAND = 10
```

```
    for i in range(CARDS_PER_HAND):
# deal 10 cards for each player
```

```
        card = deck.pop(0) # remove first
        element in deck list
```

```
        player_hand.append(card) # add
        the element to the player's hand
```

```
    return player_hand
```

```
# sort: allows players to sort their hands
for ease in viewing
```

```
def sort(cards, by="rank"):
    rank_order = ["2", "3", "4", "5", "6",
"7", "8", "9", "10", "J", "Q", "K", "A"]
    suit_order = ['♠', '♣', '♥', '♦']
```

```
    if by == "rank":
        # Sorting by rank
        cards.sort(key=lambda card:
rank_order.index(str(card.rank)))
        print(cards)
    elif by == "suit":
        # Sorting by suit
        cards.sort(key=lambda card:
suit_order.index(str(card.suit)))
        print(cards)
    else:
        raise ValueError("Invalid sorting
criteria. Use 'rank' or 'suit'.")
    return cards
```

```
# helper function
```

```
def rank_to_value(rank):
    mapping = {
        'A': 1,
        '2': 2,
        '3': 3,
        '4': 4,
        '5': 5,
        '6': 6,
        '7': 7,
        '8': 8,
        '9': 9,
        '10': 10,
        'J': 11,
        'Q': 12,
```

```
        'K': 13
    }
    return mapping.get(rank, 0)
```

```
# create_discard_pile: creates discard
pile at the beginning of the game
```

```
def create_discard_pile():
    discard_pile = [] # create a new list
    for the discard pile
```

```
    card = deck.pop(0) # remove first
    element in deck list
```

```
    discard_pile.append(card)
```

```
    return discard_pile
```

```
# draw_deck: draws a card from the deck
pile
```

```
def draw_deck(deck, hand):
    if len(deck) == 0:
```

```
        make_deck()
        card = deck.pop(0) # remove first
        element in deck
```

```
        hand.append(card) # add element to
        the hand at play
```

```
# draw_discard: draws a card from the
discard pile
```

```
def draw_discard(discard_pile, hand):
```

```
    # remove the top card (last element)
    from the discard pile
```

```
    card = discard_pile.pop()
```

```
    # add the card to the hand
```

```
    hand.append(card)
```

meld function: allows player to create melds (runs or sets) from their hand.

def meld(hand, melds):

```

    next_move = ""

    while next_move.upper() != 'B':

        print("***** NEXT MOVE
        *****")

        print("A - create run")

        print("B - create set")

    choice = input("Enter type of meld:
    ")

    # default validity is false

    valid = False

    if choice.upper() == 'A':

        # runs: consecutive values, same
        suit

        print("***** NEXT
        MOVE *****")

        print("Pick 3 or more cards to
        meld (separate with spaces).")

        print("1st card = 1, 2nd card = 2,
        etc...")

        print("Ex: 1 2 3 (1st, 2nd, and 3rd
        cards)")

        print("Your hand:", hand)

        cards = input("Enter cards:
        ").split() # get string of card indices

        # convert the list to numbers,
        added try/except block to catch non-
        integer input

        try:

            cards = list(map(int, cards))

        except ValueError:
```

```

        print("Error: please enter valid
        card numbers separated by spaces.")
```

```
        continue
```

```

        # check that at least 3 cards are
        selected, added this check
```

```
        if len(cards) < 3:
```

```

            print("Error: a meld must
            consist of at least 3 cards.")
```

```
            continue
```

```
        # check if all indices are valid
```

```

        if any(card > len(hand) or card <
        1 for card in cards):
```

```

            print("Error: invalid card
            index. please select valid cards from
            your hand.")
```

```
            continue
```

```
        # BIG CHANGES BELOW
```

```

        # check that the selected cards
        form a valid run:
```

```
        valid = True
```

```
        for i in range(len(cards) - 1):
```

```

            current_card = hand[cards[i] -
            1]
```

```

            next_card = hand[cards[i + 1] -
            1]
```

```

            if current_card.suit ==
            next_card.suit:
```

```

                # handle ace as both high
                and low (ace-king case)
```

```

                if (current_card.rank == 'A'
                and next_card.rank == '2') or \
```

```

                    (current_card.rank ==
                    'K' and next_card.rank == 'A'):
```

```
                    continue
```

```

                # check for consecutive rank
                values using the rank_to_value helper
```

```

            elif
            rank_to_value(next_card.rank) ==
            rank_to_value(current_card.rank) + 1:
```

```
                continue
```

```
            else:
```

```
                valid = False
```

```
                break
```

```
            else:
```

```
                valid = False
```

```
                break
```

```
        elif choice.upper() == 'B':
```

```
            # sets: same rank, different suits
```

```

            print("***** NEXT
            MOVE *****")
```

```

            print("Pick 3 or more cards to
            meld (separate with spaces).")
```

```

            print("1st card = 1, 2nd card = 2,
            etc...")
```

```

            print("Ex: 1 2 3 (1st, 2nd, and 3rd
            cards)")
```

```
            print("Your hand:", hand)
```

```

            cards = input("Enter cards:
            ").split() # get string of card indices
```

```

            # convert the list to numbers,
            added try/except block to catch non-
            integer input
```

```
            try:
```

```
                cards = list(map(int, cards))
```

```
            except ValueError:
```

```

                print("Error: please enter valid
                card numbers separated by spaces.")
```

```
                continue
```

```

            # check that at least 3 cards are
            selected, added this check
```

```
            if len(cards) < 3:
```

```

        print("Error: a meld must
        consist of at least 3 cards.")

        continue

    # check if all indexes are valid

    if any(card > len(hand) or card <
    1 for card in cards):

        print("Error: invalid card
        index. please select valid cards from
        your hand.")

        continue

    # check that the selected cards
    form a valid set:

    valid = True

    for i in range(len(cards) - 1):

        if hand[cards[i + 1] - 1].rank
        != hand[cards[i] - 1].rank:

            valid = False

            break

    else:

        print("Invalid input")

        continue # back to the beginning
    of the loop

    # consequences of valid and invalid
    melds

    if valid:

        print("Valid meld")

        new_meld = [] # create a new
        sublist to group the meld cards together

        # remove the chosen cards from
        hand and add them to the new meld.

        # removing in reverse order to
        avoid index shifting issues.

        # change

        for i in sorted(cards,
        reverse=True):

            card_to_add = hand[i - 1]

```

```

            new_meld.insert(0,
            card_to_add)

            hand.remove(card_to_add)

            melds.append(new_meld) #
            append the new meld (as a sublist) to the
            global melds list

            print("Updated melds:", melds)

            print("Updated hand:", hand)

        else:

            print("Error: invalid meld")

            print("Cannot add meld to
            existing melds")

    # choose next steps

    print("\n***** NEXT MOVE
    *****")

    print("What will be your next
    move?")

    print("A - create another meld")

    print("B - return to other options")

    next_move = input("Enter next
    move: ")

    # layoff: allows player to add to an
    existing meld

    def layoff(melds, hand):

        if len(melds) == 0:

            print("There are no existing melds
            to lay off on. Please try a different
            move.")

            return

        print("***** NEXT MOVE
        *****")

        print("Current melds: ", melds)

        print("Which meld would you like to
        add to? ")

        print("1st meld = 1, 2nd meld = 2,
        etc...")

```

```

    while True:

        try:

            meld_number = int(input("Enter
            meld number: "))

            if meld_number < 1 or
            meld_number > len(melds):

                print("Invalid meld number.
                Please try again.")

                continue

            break

        except ValueError:

            print("Invalid input. Please enter
            a valid integer.")

    # select meld

    selected_meld = melds[meld_number
    - 1]

    print("Selected meld: ",
    selected_meld)

    # ask player which card to lay off

    print("***** NEXT MOVE
    *****")

    print("Cards in your hand: ", hand)

    card_choice = input("Choose a card to
    lay off (enter the card number): ")

    print("1st card = 1, 2nd card = 2,
    etc...")

    try:

        chosen_card =
        hand[int(card_choice) - 1] # Get the
        selected card from the hand

        except (ValueError, IndexError):

            print("Invalid choice. Please select
            a valid card number.")

            return

    # check if it is a valid layoff

```

```

valid_layoff = False

# if meld is a set (same rank, different
suits)

    if all(card.rank ==
selected_meld[0].rank for card in
selected_meld):

        if chosen_card.rank ==
selected_meld[0].rank:

            valid_layoff = True

# if meld is a run (consecutive values,
same suit)

    if all(card.suit ==
selected_meld[0].suit for card in
selected_meld):

        first_card_rank =
rank_to_value(selected_meld[0].rank)

        last_card_rank =
rank_to_value(selected_meld[-1].rank)

        chosen_card_rank =
rank_to_value(chosen_card.rank)

# Allow layoff to the start or end of
a run

    if chosen_card_rank ==
first_card_rank - 1 or chosen_card_rank
== last_card_rank + 1:

        valid_layoff = True

if valid_layoff:

    print(f"Valid layoff! Adding
{chosen_card} to the meld.")

    selected_meld.append(chosen_card)

    hand.remove(chosen_card)

    sort(selected_meld, by="rank")

    print("Updated meld: ",
selected_meld)

    print("Updated hand: ", hand)

else:

    print("Invalid layoff. The card
cannot be added to this meld.")

# discard: allows player to discard a card

def discard(discard_pile, hand):

    print("***** NEXT MOVE
*****")

    print("Which card would you like to
discard?")

    print(hand)

    print("1st card = 1, 2nd card = 2,
etc...")

    while True:

        try:

            card_number = int(input("Enter
card number: "))

            if 1 <= card_number <=
len(hand):

                break # Valid index, exit loop.

            else:

                print(f"Please enter a number
between 1 and {len(hand)}.")

        except ValueError:

            print("Invalid input. Please enter
an integer.")

    discarded_card =
hand.pop(card_number - 1) # remove
chosen card from hand

    discard_pile.append(discarded_card)
# add discarded card to discard pile

def computer_draw(computer_hand,
discard_pile, deck):

    """

    Decide whether the computer should
draw from the discard pile or the deck.

    The computer checks if the top card of
the discard pile might help in forming a
meld.

```

If it is "useful", it draws that card;
otherwise, it draws from the deck.

```

"""

def is_card_useful(card, hand):

    """

    Determine if a given card might be
useful based on the computer's hand.

    - Useful for a set if another card of
the same rank exists.

    - Useful for a run if there's a card of
the same suit that is consecutive in rank.

    """

    # Check for set potential: same rank
exists in hand.

    for c in hand:

        if c.rank == card.rank:

            return True

    # Check for run potential: same suit
with consecutive ranks.

    card_value =
rank_to_value(card.rank)

    for c in hand:

        if c.suit == card.suit:

            if abs(rank_to_value(c.rank) -
card_value) == 1:

                return True

    return False

# Check if discard pile is non-empty

if discard_pile:

    # Assuming the top card is the last
element in the discard_pile list.

    top_card = discard_pile[-1]

    if is_card_useful(top_card,
computer_hand):

        draw_discard(discard_pile,
computer_hand)

```

```

        print(f"Computer draws
{top_card} from the discard pile.")

        return

    # Otherwise, draw from the deck.

    draw_deck(deck, computer_hand)

    print("Computer draws from the
deck.")

def computer_meld(hand):

    meld_created = False

    # ***** runs: consecutive ranks,
    same suit *****

    potential_cards = []

    for i in range(len(suits)): # the suits
are the following: ['♠', '♥', '♦', '♣']

        current_suit = suits[i] # loop
through all four suits

        for j in range(len(hand)):

            if hand[j].suit == current_suit:

                potential_cards.append(hand[j]) #
append matching suits to list

    if len(potential_cards) >= 3: # a
meld must be 3 or more cards

        unsorted_potential_ranks = []

        # find corresponding numerical
value of characters

        for x in
range(len(potential_cards)):

            unsorted_potential_ranks.append(rank_t
o_value(potential_cards[x].rank))

```

```

        # created a sorted version of the
potential ranks

        sorted_potential_ranks =
unsorted_potential_ranks.copy()

        sorted_potential_ranks.sort()

        consecutive_indexes = []

        # check to see if there are 3 or
more consecutive ranks

        for x in
range(len(sorted_potential_ranks) - 1):

            if sorted_potential_ranks[x] ==
sorted_potential_ranks[x + 1] - 1:

                consecutive_indexes.append(x)

            consecutive_indexes.append(x + 1)

            if len(consecutive_indexes)
>= 3:

                is_valid_meld = True

            else:

                consecutive_indexes = []

                is_valid_meld = False

        if is_valid_meld: # a meld must
be three or more cards

            # remove duplicates

            indexes = []

            for x in consecutive_indexes:

                if x not in indexes:

                    indexes.append(x)

            # add the meld to the melds list
and remove the cards from the
computer's hand

            new_meld = []

            for x in indexes:

                for y in
range(len(unsorted_potential_ranks)):

```

```

                    if
unsorted_potential_ranks[y] ==
sorted_potential_ranks[x]:

                        new_meld.append(potential_cards[y])

            hand.remove(potential_cards[y])

            break

            melds.append(new_meld) #
add the complete meld as one group

            print("The computer created a
new meld")

            print("Updated melds: ",
melds)

            # print("Updated computer
hand, ", hand)

            meld_created = True

        potential_cards = [] # empty the
potential cards list# if the

        else: # if there aren't more than 3
cards, a meld is not possible

            potential_cards = [] # empty the
list

            # ***** sets: same rank, different
suit *****

            ranks = [] # created empty list to hold
rank characters

            for i in range(len(hand)):

                ranks.append(hand[i].rank) # add
rank characters to empty list

            # Do not compare the last two cards
because a meld

            # must be at least 3 cards (hence the
'len(ranks) - 2'

            is_valid_meld = False

            indexes = [] # a list for valid indexes

```

```

for i in range(len(ranks) - 2):

    possible_meld = 1 # start with one
    card

    possible_index = i # start with
    current index

    for j in range(i + 1, len(ranks)):

        if ranks[i] == ranks[j]:

            possible_meld += 1 # increase
            possible meld by 1

            indexes.append(j)

    if possible_meld >= 3: # a meld
    must have at least 3 cards

        indexes.insert(0, possible_index)
        # insert first card in 1st index position

        # instead of creating separate
        melds for each card, create one meld
        from all indexes

        set_meld = [hand[i] for i in
        indexes]

        melds.append(set_meld) # add to
        melds

        print("The computer created a
        new meld")

        print("Updated melds: ", melds)
        # print updated melds

        cards_to_remove = [hand[i] for i
        in sorted(indexes, reverse=True)] # get
        cards to remove

        for card in cards_to_remove:

            hand.remove(card) # remove
            each card from computer hand

            meld_created = True

    else:

        indexes = []

    if not meld_created:

        print("The computer does not want
        to create a new meld")

```

```

def computer_layoff(computer_hand,
melds):

    if not melds: # check if melds list is
    empty

        print("No existing melds for
        computer to lay off on.")

        return

    rank_order = ["2", "3", "4", "5", "6",
    "7", "8", "9", "10", "J", "Q", "K", "A"]

    for selected_meld in melds:

        # ensure selected_meld is a list of
        cards

        if not isinstance(selected_meld, list)
        or not selected_meld:

            print(f"Error: Expected a list for
            meld, but got {type(selected_meld)}")

            continue

        for card in computer_hand[:]: #
        Iterate over a copy to allow safe removal

            valid_layoff = False

            # Check if the meld is a set (all
            cards have the same rank)

            if all(c.rank ==
            selected_meld[0].rank for c in
            selected_meld):

                if card.rank ==
                selected_meld[0].rank:

                    valid_layoff = True

            # Check if the meld is a run (all
            cards have the same suit and are
            consecutive)

            elif all(c.suit ==
            selected_meld[0].suit for c in
            selected_meld):

                # Extract ranks of the selected
                meld and sort them

```

```

        selected_ranks = [card.rank for
        card in selected_meld]

        selected_ranks.sort(key=lambda rank:
        rank_order.index(rank)) # Sort by rank
        order

        # Get first and last ranks in the
        sorted list

        first_rank = selected_ranks[0]

        last_rank = selected_ranks[-1]

        # Now check if the card fits
        before the first or after the last card

        if card.suit ==
        selected_meld[0].suit: # Check if card
        suit matches

            if

                rank_order.index(card.rank) ==
                rank_order.index(first_rank) - 1 or
                rank_order.index(

                    card.rank) ==
                    rank_order.index(last_rank) + 1:

                    valid_layoff = True

        # If a valid layoff is found,
        execute it

        if valid_layoff:

            print(f"Computer adds {card}
            to meld {selected_meld}")

            selected_meld.append(card)

        # Ensure correct order if it's a
        run

        if all(c.suit ==
        selected_meld[0].suit for c in
        selected_meld):

            selected_meld.sort(key=lambda x:
            rank_order.index(x.rank))

        computer_hand.remove(card)

```

```

        print("Updated meld:",
selected_meld)

        # print("Updated computer
hand:", computer_hand)

        return True # Successfully laid
off a card

        print("Computer could not lay off any
card.")

        return False

```

```

# import random

# # computer discard: allows the
computer to discard a card

# def computer_discard(discard_pile,
computer_hand):

#     if not computer_hand:

#         return # prevents errors if hand is
empty

#

#     random_index = random.randint(0,
len(computer_hand) - 1) # choose a
random card

#     discarded_card =
computer_hand.pop(random_index) #
remove chosen card from hand

#     discard_pile.append(discarded_card)
# add to discard pile

#

#     print(f"Computer discards
{discarded_card}")

```

```

def computer_discard(discard_pile,
computer_hand, melds):

    """

    Computer discards a card based on a
logical decision, prioritizing discarding
cards

    that do not contribute to potential
melds.

    """

```

```

    if not computer_hand:

        return # Prevents errors if hand is
empty

    # 1. Analyze hand for potential melds

    potential_melds =
analyze_potential_melds(computer_hand
, melds)

    # 2. Score cards based on their
contribution to potential melds

    card_scores = {}

    for card in computer_hand:

        card_scores[card] =
score_card(card, potential_melds)

    # 3. Choose the card with the lowest
score to discard

    discarded_card = min(card_scores,
key=card_scores.get)

    # 4. Remove the card from the hand
and add it to the discard pile

    computer_hand.remove(discarded_card)

    discard_pile.append(discarded_card)

    print(f"Computer discards
{discarded_card}")

def analyze_potential_melds(hand,
current_melds):

    """

    Analyzes the hand to identify potential
melds (runs or sets).

    Returns a dictionary of potential
melds.

    """

    potential_melds = {"runs": [], "sets":
[]}

```

```

    # Analyze for potential runs

    sorted_hand = sorted(hand,
key=lambda card:
rank_to_value(card.rank)) # helper
function needed

    for i in range(len(sorted_hand) - 2):

        if sorted_hand[i].suit ==
sorted_hand[i + 1].suit == sorted_hand[i
+ 2].suit:

            if rank_to_value(sorted_hand[i +
1].rank) ==
rank_to_value(sorted_hand[i].rank) + 1
and rank_to_value(

                sorted_hand[i + 2].rank) ==
rank_to_value(sorted_hand[i + 1].rank)
+ 1:

                potential_melds["runs"].append((sorted_
hand[i], sorted_hand[i + 1],
sorted_hand[i + 2]))

    # Analyze for potential sets

    for i in range(len(hand) - 2):

        if hand[i].rank == hand[i + 1].rank
== hand[i + 2].rank:

            potential_melds["sets"].append((hand[i],
hand[i + 1], hand[i + 2]))

    return potential_melds

def score_card(card, potential_melds):

    """

    Scores a card based on its contribution
to potential melds.

    Lower scores indicate a higher
likelihood of discarding.

    """

    score = 0

    # Check if the card is part of a
potential run

```

```

for run in potential_melds["runs"]:

    if card in run:

        score += 1 # Increment score if
card is in a potential run

    # Check if the card is part of a
potential set

    for set_cards in
potential_melds["sets"]:

        if card in set_cards:

            score += 1 # Increment score if
card is in a potential set

    # If the card is not part of any
potential meld, give it a low score

if score == 0:

    score = -1

return score

def start_turn(player_hand):

    user_input = input("Your turn: ")

    if user_input == "":

        print("Your hand: ", player_hand) #
~~~~~

        # print()

    elif user_input.lower() ==
"instructions":

        print(instructions)

    elif user_input.lower() == "sort by
rank":

        sort(player_hand, by="rank")

    elif user_input.lower() == "sort by
suit":

        sort(player_hand, by="suit")

    elif user_input.lower() == "shuffle" or
user_input.lower() == "shuffle deck":

        shuffle(deck)

```

```

def computer_turn(computer_hand,
melds, discard_pile, deck):

    print("\n***** COMPUTER'S
TURN *****")

    computer_draw(computer_hand,
discard_pile, deck)

    computer_meld(computer_hand)

    computer_layoff(computer_hand,
melds)

    computer_discard(discard_pile,
computer_hand, melds)

    print() # blank space

def play_game():

    # ***** Game Code ***** #

    # create the deck

    make_deck()

    # shuffle the deck

    shuffle(deck)

    # deal each player a hand of cards

    computer_hand = deal(deck)

    player_hand = deal(deck)

    # create the discard pile

    discard_pile = create_discard_pile()

    print("Discard Pile: ", discard_pile)

    # print new deck

    # print("Deck: ", deck)

    # print("Computer hand: ",
computer_hand)

    # print("Player hand: ", player_hand)

```

```

# Game starts here

keep_playing = True

while keep_playing:

    start_turn(player_hand)

    # ***** Drawing a card ***** #

    # choices for picking up a card

    # print discard pile (deck pile is
hidden because deck is face down and
unknown to the player

    # print("Deck: {}".format(deck[-
1]))

    print("Discard Pile:
{}".format(discard_pile[-1]))

    print("***** NEXT MOVE
*****")

    print("Your hand: ", player_hand)

    print("A - Draw from deck pile")

    print(f"B - Draw from discard pile
(Pick up the {discard_pile[-1]})\n")

    while True:

        choice = input("Enter choice
(A/B): ") # computer or user makes
choice

        if choice.upper() in ['A', 'B']:

            break

        print("Invalid input. Please enter
'A' or 'B'.")

    if choice.upper() == 'A':

        draw_deck(deck, player_hand)

    elif choice.upper() == 'B':

        draw_discard(discard_pile,
player_hand)

    else:

        print("Invalid input.")

```

```

print("Your hand: ", player_hand)

# ***** Meld/Layoff and Discard
***** #

# changed MELD and LAYOFF to
separate choices

while True:

    # placed options inside the loop

    print(f"Current melds:
{melds}\n") # ~~~~~

    # placed options inside the loop

    print("***** NEXT
MOVE *****")

    print("A - Meld")

    print("B - Layoff")

    print("C - Discard a card")

    choice = input("Enter choice
(A/B/C): ")

    if choice.upper() == 'A':

        meld(player_hand, melds)

    elif choice.upper() == 'B':

        layoff(melds, player_hand)

    elif choice.upper() == 'C':

        discard(discard_pile,
player_hand)

        # check to see if player won

        if len(player_hand) == 0:

            keep_playing = False

            break

        computer_turn(computer_hand, melds,
discard_pile, deck)

        # check to see if computer won

        if len(computer_hand) == 0:

            keep_playing = False

            break

        else:

            print("Invalid input. Please
enter 'A' or 'B' or 'C'.")

    # ***** Win Detection ***** #

    #if len(computer_hand) == 0 or
len(player_hand) == 0:

        #keep_playing = False

        break

    if len(computer_hand) == 0:

        print("\nYou lose :(")

        global computer_score

        computer_score +=
len(player_hand)

        if len(player_hand) == 0:

            print("\nYou win! :)")

            global player_score

            player_score +=
len(computer_hand)

    # Print results

    print("***** GAME RESULTS
*****")

    print("Your Score: ", player_score)

    print("Computer Score: ",
computer_score)

play_game()

while input("Do you want to play again?
(yes/no)") == "yes":

    melds = []

    play_g

```